

Triagem paralela de editais de compra pública

070080 Sistemas Distribuídos e Paralelos · Etapa 1 do Projeto de Solução Distribuída · Turma CC6MA · Prof. Fábio Rocha de Araújo · 22/09
Equipe: Yuri Aguiar (24070309) · Pedro Paulo (24070313) · João Rath (24070338) · Fábio Gabriel (21070209)

1. O problema

O Portal Nacional de Contratações Públicas publica dezenas de milhares de editais por mês. Uma empresa que vende software para o setor público precisa separar, dentro desse volume, os editais que têm a ver com as suas linhas de produto — e precisa fazer isso antes de a sessão pública abrir. A triagem manual não acompanha.

A aplicação recebe um lote de editais em texto e, para cada um, executa quatro etapas: normalização Unicode com remoção de acentos, tokenização com descarte de palavras vazias, pontuação de aderência contra uma taxonomia de termos com peso — positivos para software de gestão e arrecadação, negativos para obra, merenda e combustível — e o cálculo de uma assinatura MinHash de 64 posições sobre os *shingles* de cinco tokens da seção do objeto. A assinatura serve para reconhecer o mesmo edital republicado por municípios diferentes, caso frequente no PNCP. A saída é um relatório com o ranking dos editais mais aderentes, a distribuição por linha de produto, a frequência dos termos da taxonomia e a contagem de republicações.

1.1 Por que o trabalho se divide

A unidade de trabalho é **um edital**. O processamento de um edital não lê nem escreve nada de outro edital: as funções de `nucleo.py` são puras e o módulo não tem estado global. A dependência aparece só na agregação, e a operação de agregação é associativa e comutativa, o que permite que cada fluxo acumule um parcial próprio e funda esse parcial no total ao fim de cada lote.

Condição exigida	Como a aplicação atende
Partes independentes	Um edital por unidade. Sem leitura ou escrita cruzada entre unidades.
Volume suficiente	8000 documentos, 81 MiB de texto. Tempo sequencial medido: 174,29 s.
Resultado verificável	O relatório final é canonizado (ordenação determinada só pelo conteúdo) e reduzido a um SHA-256. As duas versões têm de produzir a mesma impressão digital.
Estado escrito por mais de um fluxo	O agregador global: contadores, dicionário de termos, ranking e mapa de assinaturas, em memória compartilhada.

2. Estratégia de paralelização

Decisão	Escolha	Justificativa
Tipo de paralelismo	Dados	A mesma sequência de operações se aplica a documentos diferentes. Não há estágios distintos que possam correr em paralelo, então paralelismo de tarefas não se aplica.
Processo ou thread	Processo	O trabalho é limitado por processador, não por espera: a leitura do arquivo responde por menos de 0,5 % do tempo; o resto é aritmética de inteiros e hashing em Python puro. Em CPython o GIL impede que duas threads executem bytecode ao mesmo tempo, de modo que a versão com threads mediria speedup próximo de 1. A aplicação tem o modo <code>--motor thread</code> justamente para mostrar isso na medição.
Número de trabalhadores	1 e 2	Varre-se de 1 até o número de vCPU da instância, para que a curva de speedup mostre onde o ganho satura e por quê.
Divisão do trabalho	Fila comum de lotes	O corpus é cortado em lotes de 150 documentos, postos numa única fila. Cada trabalhador puxa o próximo lote quando termina o anterior. É balanceamento dinâmico: um trabalhador que pegue um lote caro não atrasa os outros, ao contrário do que aconteceria com uma divisão fixa em <i>n</i> fatias.

3. A seção crítica e a primitiva que a protege

O estado compartilhado é o agregador global, mantido por um `multiprocessing.Manager` e escrito por todos os trabalhadores. A seção crítica é a função `fundir_no_global` de `src/paralelo.py`, e a primitiva é um **`multiprocessing.Lock`** — exclusão mútua simples, porque o recurso admite um escritor por vez e não há condição de espera a sinalizar, o que dispensaria semáforo contado ou `monitor`.

```

lock.acquire()
try:
    relógio = max(relógio_local, estado["relógio"]) + 1 # carimbo de Lamport
    estado["relógio"] = relógio
    estado["documentos"] = estado["documentos"] + parcial["documentos"]
    termos = estado["termos"]; ...; estado["termos"] = termos
    for chave, doc_id in parcial["assinaturas"].items(): # verificar-entao-agir
        anterior = assinaturas.get(chave)
        if anterior is None: assinaturas[chave] = doc_id
        else: duplicados += 1
    eventos.append({"quem": identificador, "o_que": "fusao", ...})
finally:
    lock.release()

```

Duas corridas distintas vivem aí. A primeira é a *atualização perdida*: estado["documentos"] é lido, somado e gravado em três passos, e entre o primeiro e o terceiro cabe a gravação de outro trabalhador, que se perde. A segunda é o *verificar-entao-agir* sobre o mapa de assinaturas: dois trabalhadores perguntam se a mesma assinatura já existe, ambos recebem "não", e uma republicação deixa de ser contada.

3.1 Por que o laço inteiro fica fora da trava

O processamento dos documentos de um lote acontece **antes** de `acquire`, num agregador parcial local ao processo. Só a fusão do parcial entra na trava. Uma trava em volta do laço inteiro também seria correta, mas serializaria todo o trabalho e o speedup cairia para perto de 1. A aplicação traz essa versão no modo `--sincronizacao larga`, e ela é medida junto com as outras justamente para tornar a diferença visível.

3.2 Troca de mensagem e carimbo lógico

Há troca de mensagem: a fila de lotes e o canal até o processo do Manager. Como os processos não compartilham relógio físico, a ordem dos eventos é registrada por um **relógio lógico de Lamport**. Cada trabalhador mantém um contador local, incrementado ao receber um lote; ao fundir, adianta o contador para `max(local, global) + 1`, grava esse valor no estado e registra o evento. Cada registro contém *quem* (identificador do trabalhador), *o quê* (fusão no agregador global), *carimbo lógico* e *sobre o quê* (número do lote e quantidade de documentos). O registro completo sai em resultados/eventos.json e permite reconstruir uma ordenação consistente das fusões sem relógio comum. A execução arquivada registrou **54** fusões de 2 trabalhadores, com carimbo lógico máximo 59.

3.3 Como a correção é demonstrada

`src/verificar.py` roda a versão sequencial uma vez e a paralela várias vezes sobre a mesma entrada e compara os SHA-256. Com `--modo lock` as execuções coincidem entre si e com a sequencial. Com `--modo nenhuma` a mesma entrada produz impressões digitais diferentes a cada execução: é a condição de corrida exibida ao vivo, e não uma descrição dela.

4. Recursos provisionados

Decisão	Escolha	Justificativa
Região e zonas	us-east-1 (Norte da Virgínia), uma zona (us-east-1c)	Instância isolada em zona única: vale o compromisso de 99,5 % do SLA do EC2, ou 216 minutos de indisponibilidade admitida em 30 dias. Distribuir por duas ou mais zonas elevaria o compromisso para 99,99 % (4,3 minutos), mas o experimento exige que T(1) e T(n) sejam medidos no <i>mesmo</i> hardware: duas instâncias em zonas diferentes não comparariam a mesma coisa. A disponibilidade não é requisito de um lote que roda sob demanda.
Família, tamanho e quantidade	1 × c5.large	Família otimizada para computação, sem créditos de CPU. Instância burstável (t2, t3) é inadequada para medir speedup: ao esgotar os créditos, o desempenho cai no meio da medição e o T(n) deixa de ser comparável ao T(1).
Entrada e saída	EBS gp3 30 GiB; saída versionada no repositório	O corpus (81 MiB) é gerado no volume local, para que a leitura não introduza latência de rede no tempo medido. A saída são dois arquivos JSON de poucos KiB — a medição e o registro de eventos — versionados no repositório junto com o código que os gerou. Um bucket S3 resolveria o mesmo problema, sobreviver ao encerramento da instância, com um serviço a mais para provisionar e controlar: para este volume, o repositório basta e mantém resultado e código na mesma versão.
Portas e origem	22/tcp de 177.180.12.98/32; 8000/tcp de 0.0.0.0/0	A porta administrativa não aceita 0.0.0.0/0: a porta 22 exposta à internet recebe tentativas automáticas de autenticação continuamente, e restringi-la ao endereço da equipe retira a instância dessa superfície sem depender da força da chave. A porta 8000 serve o painel da aplicação e é a única aberta ao público.
Ciclo de vida	Ligada durante a janela de medição	Criar a instância por execução traria hardware subjacente potencialmente diferente entre T(1) e T(n). Como o speedup exige a mesma máquina, a instância permanece ligada do início da medição até o fim da apresentação, e é encerrada depois.

5. Medição

Tempo sequencial e tempo paralelo medidos na mesma máquina (Intel(R) Xeon(R) Platinum 8124M CPU @ 3.00GHz, 2 núcleos lógicos), com a mesma entrada de 8000 documentos, com 3 repetições por configuração; as tabelas trazem a mediana. O T(1) vem da versão sequencial, não da paralela com um trabalhador, que já carrega o custo da fila e do Manager.

Motor	n	T(n) (s)	Speedup	Teto de Amdahl	Eficiência	e (Karp-Flatt)	Estável	= sequencial
sequencial	1	174,29	1,00	1,00	1,00	—	sim	referência
processo	1	177,71	0,98	1,00	0,98	—	sim	sim
processo	2	171,47	1,02	2,00	0,51	0,9676	sim	sim
thread	2	176,56	0,99	2,00	0,49	1,0260	sim	sim

Estável: as repetições daquela configuração devolveram o mesmo SHA-256 entre si. = **sequencial:** esse SHA-256 é também o da versão sequencial — a paralelização não mudou o resultado, só o tempo. As duas colunas são verificadas a cada execução da bancada, não afirmadas no texto.

A fração paralelizável $p = 0,9977$ não foi arbitrada: a versão sequencial cronometra separadamente a leitura do arquivo e o trabalho por documento, e p é a razão entre o segundo e o total.

6. O que limitou o ganho

O melhor resultado foi com $n = 2$: o tempo caiu de 174,29 s para 171,47 s, um speedup medido de **1,02** contra um teto de Amdahl de 2,00 para a fração paralelizável instrumentada $p = 0,9977$. A diferença é de 0,98 (49,1 %). A métrica de Karp-Flatt devolve uma fração serial experimental de $e = 0,9676$, acima do $(1 - p) = 0,0023$ medido por instrumentação: a diferença entre as duas é exatamente o custo que a paralelização introduziu e que não existia na versão sequencial.

- **vCPU não são núcleos independentes.** A instância tem 2 vCPU sobre um único núcleo físico. Os dois são threads de hardware da mesma unidade de execução: para trabalho limitado por processador, que ocupa a unidade o tempo todo, o segundo trabalhador não tem onde executar. A lei de Amdahl supõe n processadores independentes, e essa hipótese não vale aqui — e **é a causa dominante** da diferença.
- **Comunicação entre processos.** O agregador global vive no processo do Manager; cada leitura e cada escrita de estado[...] é uma ida e volta por soquete, com serialização. Esse custo não existe na versão sequencial, logo não aparece em p e o teto de Amdahl não o prevê. É o motivo de a assinatura viajar comprimida em 16 dígitos em vez das 64 posições originais.
- **Divisão desigual do trabalho.** Os lotes têm o mesmo número de documentos, mas os documentos têm tamanhos diferentes, e o custo do MinHash cresce com o número de shingles. No fim da execução alguns trabalhadores já esvaziaram a fila enquanto outros ainda processam o último lote. A fila comum reduz esse efeito, mas não o elimina: o limite inferior é o lote mais caro.
- **Espera na seção crítica.** A espera acumulada na trava foi de 0,01 s, 0,00 % do tempo total de trabalhador (342,93 s). É a menor de todas: cada trabalhador entra na trava uma vez por lote, não uma vez por documento. Não é aqui que o ganho se perde.

Threads, para comparação. A mesma aplicação com threading e $n = 2$ mediu speedup de 0,99 (176,56 s). O trabalho é limitado por processador e o GIL serializa a execução de bytecode: as threads existem, mas apenas uma avança por vez. É a evidência de que a escolha por processos não foi estilística.